Lab: Deploy Azure Databricks in a Hub-and-Spoke Network with Azure Firewall using Terraform

Description

In this lab, you will deploy an Azure Databricks workspace into a **hub-and-spoke network** using Terraform (AzureRM v5.6.0). The Databricks workspace lives in a **spoke VNet**. All outbound traffic from its clusters is forced through an **Azure Firewall** in a central **hub VNet**.

In the VNet injection lab, a NAT Gateway let clusters reach **anything** on the internet. Here, the firewall allows only what Databricks needs:

- The Databricks **control plane**
- Databricks **artifact and log storage**
- Approved **package repositories** (PyPI, Maven, CRAN)

Everything else is **denied and logged**.

This is how most regulated enterprises, such as banks and healthcare providers, run Databricks. It prevents data exfiltration, gives security teams one place to control and audit egress, and lets many spokes share a single firewall.

This lab builds on the VNet injection lab. Private Link is left out so the focus stays on egress control.

Learning Objectives

By the end of this lab, you will be able to:

1. Build a **hub VNet** with an **Azure Firewall** and a **Firewall Policy**.
2. Write **network rules** using **service tags** and **application rules** using **FQDNs**.
3. Build a **spoke VNet** for Databricks and connect it to the hub with **VNet peering**.
4. Force all cluster egress through the firewall with a **route table (UDR)**.

5. Send firewall logs to **Log Analytics** and query allowed and denied traffic.
6. Prove from a running cluster that approved traffic works and everything else is blocked.

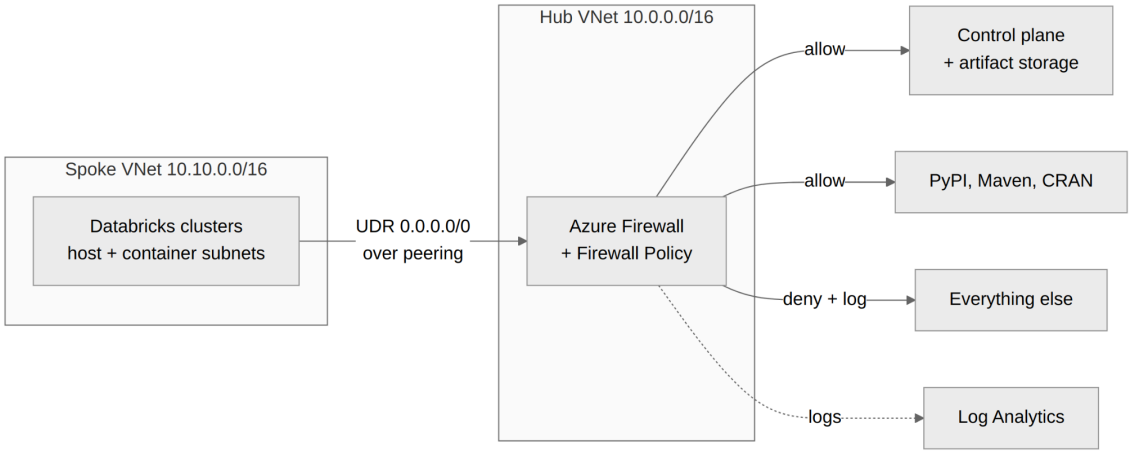
Prerequisites

Make sure you have the following before starting:

- A valid **Azure subscription** where you have **Contributor** (or Owner) access.
- **Terraform** and **Azure CLI** installed locally.
- **Visual Studio Code (VS Code)** or any IDE.
- Completion of the previous lab, *Deploy Azure Databricks with VNet Injection using Terraform*.

Azure Firewall is billed per hour while it exists, even when idle. Complete the lab in one sitting and run the clean-up step as soon as you finish.

Architecture



The route table sends all spoke traffic leaving the VNet to the firewall's private IP. The firewall checks each connection against its policy, then allows it out through its own public IP or denies it.

What the Firewall Allows

Traffic	Rule type	Destination	Ports
Databricks control plane (web app, secure cluster connectivity relay)	Network	AzureDatabricks service tag	443, 3306, 8443-8451
Artifact, log and DBFS root storage	Network	Storage.WestEurope service tag	443
Legacy Hive metastore	Network	Sql.WestEurope service tag	3306
Log and telemetry delivery	Network	EventHub.WestEurope service tag	9093
Python packages	Application	pypi.org, pythonhosted.org	443
Java and Scala packages	Application	repo.maven.apache.org, repo1.maven.org	443
R packages	Application	cran.r-project.org	443
Anything else	None	Any	Denied

Service tags are Microsoft-maintained groups of IP ranges. Azure updates them automatically, so you do not have to track Databricks IP addresses yourself.

Step by Step Guide

Step 1: Authenticate to Azure

```
az login
az account set --subscription "<your-subscription-id>"
```

Step 2: Create the Project and Configure the Provider

Create a new folder called databricks_hub_spoke. Inside it, create a file named providers.tf:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "5.6.0"
    }
  }
}

provider "azurerm" {
  features {}
  subscription_id = "<your-subscription-id>"

  resource_providers_to_register = [
    "Microsoft.Databricks",
    "Microsoft.Network",
    "Microsoft.OperationalInsights",
    "Microsoft.Insights",
  ]
}
```

Explanation:

- Microsoft.OperationalInsights → needed for the Log Analytics workspace.
- Microsoft.Insights → needed for the firewall's diagnostic settings.

Initialize the project:

```
terraform init
```

Step 3: Define Variables and Values

Create a file named variables.tf:

```
# ---------- General ----------
variable "location" {
  type    = string
  default = "West Europe"
```

```
}

variable "service_tag_region" {
  type        = string
  description = "Region name used in regional service tags, e.g. WestEurope"
  default     = "WestEurope"
}

variable "tags" {
  type = map(string)
}

# ---------- Hub ----------
variable "hub_resource_group_name" {
  type = string
}

variable "hub_vnet_name" {
  type = string
}

variable "hub_vnet_address_space" {
  type    = list(string)
  default = ["10.0.0.0/16"]
}

variable "firewall_subnet_prefix" {
  type    = list(string)
  default = ["10.0.1.0/26"]
}

variable "firewall_name" {
  type = string
}

# ---------- Spoke ----------
variable "spoke_resource_group_name" {
```

```
  type = string
}

variable "spoke_vnet_name" {
  type = string
}

variable "spoke_vnet_address_space" {
  type    = list(string)
  default = ["10.10.0.0/16"]
}

variable "public_subnet_prefix" {
  type    = list(string)
  default = ["10.10.1.0/24"]
}

variable "private_subnet_prefix" {
  type    = list(string)
  default = ["10.10.2.0/24"]
}

# ---------- Workspace ----------
variable "workspace_name" {
  type = string
}

variable "managed_resource_group_name" {
  type = string
}
```

Create a file named terraform.tfvars:

```
location                  = "West Europe"
service_tag_region        = "WestEurope"
hub_resource_group_name   = "rg-hub-network-lab"
hub_vnet_name             = "vnet-hub"
```

```
firewall_name               = "afw-hub"
spoke_resource_group_name   = "rg-spoke-databricks-lab"
spoke_vnet_name             = "vnet-spoke-databricks"
workspace_name              = "dbw-hub-spoke-lab"
managed_resource_group_name = "rg-spoke-databricks-lab-managed"
tags = {
  environment = "dev"
  project     = "databricks-hub-spoke-lab"
}
```

If you change location, change service_tag_region to match, for example East US and EastUS.

Step 4: Create the Resource Groups

In most enterprises, a central network team owns the hub and application teams own the spokes. Separate resource groups reflect that split.

Create a file named resource_groups.tf:

```
resource "azurerm_resource_group" "hub" {
  name     = var.hub_resource_group_name
  location = var.location
  tags     = var.tags
}

resource "azurerm_resource_group" "spoke" {
  name     = var.spoke_resource_group_name
  location = var.location
  tags     = var.tags
}
```

Step 5: Create the Hub VNet

Create a file named hub_network.tf:

```
resource "azurerm_virtual_network" "hub" {
  name                = var.hub_vnet_name
  address_space       = var.hub_vnet_address_space
```

```
  location            = azurerm_resource_group.hub.location
  resource_group_name = azurerm_resource_group.hub.name
  tags                = var.tags
}

resource "azurerm_subnet" "firewall" {
  name                 = "AzureFirewallSubnet"
  resource_group_name  = azurerm_resource_group.hub.name
  virtual_network_name = azurerm_virtual_network.hub.name
  address_prefixes     = var.firewall_subnet_prefix
}
```

Explanation:

- AzureFirewallSubnet → the subnet name must be exactly this. Azure Firewall will not deploy into any other subnet.
- /26 → the minimum size Azure Firewall requires, so it can scale out.

Step 6: Create the Azure Firewall

Create a file named firewall.tf:

```
resource "azurerm_public_ip" "firewall" {
  name                = "pip-${var.firewall_name}"
  location            = azurerm_resource_group.hub.location
  resource_group_name = azurerm_resource_group.hub.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = var.tags
}

resource "azurerm_firewall_policy" "hub" {
  name                     = "afwp-hub"
  location                 = azurerm_resource_group.hub.location
  resource_group_name      = azurerm_resource_group.hub.name
  sku                      = "Standard"
  threat_intelligence_mode = "Alert"
  tags                     = var.tags
```

```
}

resource "azurerm_firewall" "hub" {
  name                = var.firewall_name
  location            = azurerm_resource_group.hub.location
  resource_group_name = azurerm_resource_group.hub.name
  sku_name            = "AZFW_VNet"
  sku_tier            = "Standard"
  firewall_policy_id  = azurerm_firewall_policy.hub.id

  ip_configuration {
    name                 = "ipconfig-firewall"
    subnet_id            = azurerm_subnet.firewall.id
    public_ip_address_id = azurerm_public_ip.firewall.id
  }

  tags = var.tags
}
```

Explanation:

- Firewall Policy → holds the rules separately from the firewall. One policy can be shared by many firewalls.
- Public IP → all allowed outbound traffic leaves Azure from this single address. External partners can allowlist it.
- threat_intelligence_mode = "Alert" → logs traffic to known malicious addresses. Production setups often use Deny.

Step 7: Define the Firewall Rules

Create a file named firewall_rules.tf:

```
resource "azurerm_firewall_policy_rule_collection_group" "databricks" {
  name               = "rcg-databricks"
  firewall_policy_id = azurerm_firewall_policy.hub.id
  priority           = 100

  network_rule_collection {
```

```
  name     = "databricks-platform"
  priority = 100
  action   = "Allow"

  rule {
    name                 = "databricks-control-plane"
    protocols            = ["TCP"]
    source_addresses     = var.spoke_vnet_address_space
    destination_addresses = ["AzureDatabricks"]
    destination_ports    = ["443", "3306", "8443-8451"]
  }

  rule {
    name                 = "artifact-storage"
    protocols            = ["TCP"]
    source_addresses     = var.spoke_vnet_address_space
    destination_addresses = ["Storage.${var.service_tag_region}"]
    destination_ports    = ["443"]
  }

  rule {
    name                 = "hive-metastore"
    protocols            = ["TCP"]
    source_addresses     = var.spoke_vnet_address_space
    destination_addresses = ["Sql.${var.service_tag_region}"]
    destination_ports    = ["3306"]
  }

  rule {
    name                 = "event-hub-logs"
    protocols            = ["TCP"]
    source_addresses     = var.spoke_vnet_address_space
    destination_addresses = ["EventHub.${var.service_tag_region}"]
    destination_ports    = ["9093"]
  }
}
```

```
application_rule_collection {
  name     = "package-repositories"
  priority = 200
  action   = "Allow"

  rule {
    name             = "python-pypi"
    source_addresses  = var.spoke_vnet_address_space
    destination_fqdns = ["pypi.org", "*.pypi.org", "pythonhosted.org", "*.pythonhosted.org"]

    protocols {
      type = "Https"
      port = 443
    }
  }

  rule {
    name             = "java-maven"
    source_addresses  = var.spoke_vnet_address_space
    destination_fqdns = ["repo.maven.apache.org", "repo1.maven.org"]

    protocols {
      type = "Https"
      port = 443
    }
  }

  rule {
    name             = "r-cran"
    source_addresses  = var.spoke_vnet_address_space
    destination_fqdns = ["cran.r-project.org"]

    protocols {
      type = "Https"
      port = 443
    }
  }
```

```
  }
}
```

Explanation:

- Network rules → match on IP address, port and protocol. Service tags such as AzureDatabricks cover Microsoft-managed IP ranges.
- Application rules → match on the domain name (FQDN) in HTTPS traffic. Use them for third-party sites such as PyPI.
- source_addresses → only traffic from the spoke VNet matches these rules.
- Default deny → Azure Firewall blocks anything that no rule allows. You never write a "deny all" rule.

The Storage service tag covers every storage account in the region. For tighter control, replace it with the exact Databricks artifact and log storage domains for your region from the Azure Databricks documentation.

Step 8: Send Firewall Logs to Log Analytics

Create a file named firewall_logs.tf:

```
resource "azurerm_log_analytics_workspace" "hub" {
  name                = "log-hub-firewall"
  location            = azurerm_resource_group.hub.location
  resource_group_name = azurerm_resource_group.hub.name
  sku                 = "PerGB2018"
  retention_in_days   = 30
  tags                = var.tags
}

resource "azurerm_monitor_diagnostic_setting" "firewall" {
  name                           = "diag-firewall"
  target_resource_id             = azurerm_firewall.hub.id
  log_analytics_workspace_id     = azurerm_log_analytics_workspace.hub.id
  log_analytics_destination_type = "Dedicated"

  enabled_log {
    category = "AZFWNetworkRule"
```

```
  }

  enabled_log {
    category = "AZFWApplicationRule"
  }
}
```

Explanation:

- Dedicated → writes each log type to its own table (AZFWNetworkRule, AZFWApplicationRule), which is easier to query than one shared table.

Step 9: Create the Spoke Network

This is the same Databricks network as the VNet injection lab, without the NAT Gateway.

Create a file named spoke_network.tf:

```
locals {
  databricks_subnets = {
    host      = { name = "snet-databricks-host", prefix = var.public_subnet_prefix }
    container = { name = "snet-databricks-container", prefix = var.private_subnet_prefix }
  }
}

resource "azurerm_virtual_network" "spoke" {
  name                = var.spoke_vnet_name
  address_space       = var.spoke_vnet_address_space
  location            = azurerm_resource_group.spoke.location
  resource_group_name = azurerm_resource_group.spoke.name
  tags                = var.tags
}

resource "azurerm_subnet" "databricks" {
  for_each = local.databricks_subnets

  name                 = each.value.name
  resource_group_name  = azurerm_resource_group.spoke.name
```

```
  virtual_network_name = azurerm_virtual_network.spoke.name
  address_prefixes     = each.value.prefix

  delegation {
    name = "databricks-${each.key}-delegation"

    service_delegation {
      name = "Microsoft.Databricks/workspaces"
      actions = [
        "Microsoft.Network/virtualNetworks/subnets/join/action",
        "Microsoft.Network/virtualNetworks/subnets/prepareNetworkPolicies/action",
        "Microsoft.Network/virtualNetworks/subnets/unprepareNetworkPolicies/action",
      ]
    }
  }
}

resource "azurerm_network_security_group" "spoke" {
  name                = "nsg-spoke-databricks"
  location            = azurerm_resource_group.spoke.location
  resource_group_name = azurerm_resource_group.spoke.name
  tags                = var.tags
}

resource "azurerm_subnet_network_security_group_association" "databricks" {
  for_each = azurerm_subnet.databricks

  subnet_id                 = each.value.id
  network_security_group_id = azurerm_network_security_group.spoke.id
}
```

Step 10: Peer the Spoke and Hub VNets

Peering is always created in **both directions**. Create a file named **peering.tf**:

```
resource "azurerm_virtual_network_peering" "spoke_to_hub" {
  name                         = "peer-spoke-to-hub"
  resource_group_name          = azurerm_resource_group.spoke.name
```

```
  virtual_network_name         = azurerm_virtual_network.spoke.name
  remote_virtual_network_id    = azurerm_virtual_network.hub.id
  allow_virtual_network_access = true
  allow_forwarded_traffic      = true
}

resource "azurerm_virtual_network_peering" "hub_to_spoke" {
  name                         = "peer-hub-to-spoke"
  resource_group_name          = azurerm_resource_group.hub.name
  virtual_network_name         = azurerm_virtual_network.hub.name
  remote_virtual_network_id    = azurerm_virtual_network.spoke.id
  allow_virtual_network_access = true
  allow_forwarded_traffic      = true
}
```

Explanation:

- allow_forwarded_traffic → lets traffic that the firewall forwards on behalf of the spoke cross the peering.
- If only one side is created, the peering stays Initiated and no traffic flows.

Step 11: Route Spoke Traffic Through the Firewall

Create a file named routes.tf:

```
resource "azurerm_route_table" "spoke" {
  name                          = "rt-spoke-databricks"
  location                      = azurerm_resource_group.spoke.location
  resource_group_name           = azurerm_resource_group.spoke.name
  bgp_route_propagation_enabled = false
  tags                          = var.tags
}

resource "azurerm_route" "default_to_firewall" {
  name                   = "default-to-firewall"
  resource_group_name    = azurerm_resource_group.spoke.name
  route_table_name       = azurerm_route_table.spoke.name
  address_prefix         = "0.0.0.0/0"
```

```
  next_hop_type          = "VirtualAppliance"
  next_hop_in_ip_address = azurerm_firewall.hub.ip_configuration[0].private_ip_address
}

resource "azurerm_subnet_route_table_association" "databricks" {
  for_each = azurerm_subnet.databricks

  subnet_id      = each.value.id
  route_table_id = azurerm_route_table.spoke.id
}
```

Explanation:

- 0.0.0.0/0 → VirtualAppliance → overrides Azure's default internet route. Every packet leaving the VNet goes to the firewall's private IP first.
- Traffic inside the spoke VNet, such as driver-to-worker traffic, still flows directly and does not touch the firewall.
- bgp_route_propagation_enabled = false → stops routes learned from VPN or ExpressRoute gateways from bypassing the firewall.

Step 12: Create the Databricks Workspace

Create a file named databricks_workspace.tf:

```
resource "azurerm_databricks_workspace" "workspace" {
  name                        = var.workspace_name
  resource_group_name         = azurerm_resource_group.spoke.name
  location                    = azurerm_resource_group.spoke.location
  sku                         = "premium"
  managed_resource_group_name = var.managed_resource_group_name

  custom_parameters {
    no_public_ip        = true
    virtual_network_id  = azurerm_virtual_network.spoke.id
    public_subnet_name  = azurerm_subnet.databricks["host"].name
    private_subnet_name = azurerm_subnet.databricks["container"].name
```

```
    public_subnet_network_security_group_association_id  =
azurerm_subnet_network_security_group_association.databricks["host"].id
    private_subnet_network_security_group_association_id =
azurerm_subnet_network_security_group_association.databricks["container"].id
  }

  tags = var.tags

  depends_on = [
    azurerm_subnet_route_table_association.databricks,
    azurerm_virtual_network_peering.spoke_to_hub,
    azurerm_virtual_network_peering.hub_to_spoke,
    azurerm_firewall_policy_rule_collection_group.databricks,
  ]
}
```

Explanation:

- depends_on → the routes, peering and firewall rules must all be in place before clusters can start. Otherwise the first cluster fails because its traffic has nowhere to go.

Step 13: Define Outputs

Create a file named outputs.tf:

```
output "databricks_workspace_url" {
  value = "https://${azurerm_databricks_workspace.workspace.workspace_url}"
}

output "firewall_private_ip" {
  value = azurerm_firewall.hub.ip_configuration[0].private_ip_address
}

output "firewall_public_ip" {
  value = azurerm_public_ip.firewall.ip_address
}
```

```
output "log_analytics_workspace" {
  value = azurerm_log_analytics_workspace.hub.name
}
```

Step 14: Deploy the Infrastructure

```
terraform fmt
terraform validate
terraform plan
terraform apply --auto-approve
```

Deployment takes about 15 to 20 minutes. Azure Firewall alone takes around 10 minutes.

View the outputs:

```
terraform output
```

Step 15: Verify the Traffic Controls

15.1 Check the network in the Azure Portal

1. Open vnet-spoke-databricks → Peerings. The peering status is Connected. Check the same in vnet-hub.
2. Open rt-spoke-databricks. It has one route, 0.0.0.0/0, with next hop Virtual appliance at the firewall_private_ip. Under Subnets, both Databricks subnets are listed.
3. Open afwp-hub → Rule collections. You see databricks-platform (network) and package-repositories (application).

15.2 Start a cluster

1. Open the databricks_workspace_url and sign in.
2. Go to Compute → Create compute. Choose Single node, the latest LTS runtime and the smallest node type. Set Terminate after to 20 minutes.

3. Click **Create compute** and wait for the state to change to **Running**.

A running cluster proves the control plane and storage rules work. Every packet it needed passed through the firewall.

15.3 Test allowed and blocked traffic

Create a notebook attached to the cluster and run each command in its own cell:

```
%pip install tabulate
```

```
%sh curl -sS -m 10 -o /dev/null -w "%{http_code}\n" https://pypi.org/simple/
```

```
%sh curl -sS -m 10 -o /dev/null -w "%{http_code}\n" https://www.google.com
```

```
%sh curl -sS -m 10 -o /dev/null -w "%{http_code}\n" https://github.com
```

Command	Expected result	Why
pip install tabulate	Installs successfully	PyPI is allowed by the **python-pypi** rule
curl pypi.org	200	Same rule
curl google.com	Connection error	No rule allows it, so the firewall denies it
curl github.com	Connection error	Also not allowed, even though it is a common developer site

15.4 Query the firewall logs

In the Azure Portal, open **log-hub-firewall** → **Logs** and run:

```
AZFWApplicationRule
| where TimeGenerated > ago(1h)
| project TimeGenerated, SourceIp, Fqdn, Action, Rule
| order by TimeGenerated desc
```

You see Allow entries for pypi.org and Deny entries for www.google.com and github.com, each with the cluster's private IP as the source.

To see control-plane traffic, run:

```
AZFWNetworkRule
| where TimeGenerated > ago(1h)
| summarize Connections = count() by DestinationIp, DestinationPort, Action, Rule
| order by Connections desc
```

Logs can take 5 to 10 minutes to appear.

15.5 Challenge: open a new destination

A data scientist needs to clone code from GitHub. Add this rule inside the application_rule_collection block in firewall_rules.tf:

```
  rule {
    name             = "github"
    source_addresses  = var.spoke_vnet_address_space
    destination_fqdns = ["github.com", "*.github.com", "*.githubusercontent.com"]

    protocols {
      type = "Https"
      port = 443
    }
  }
```

Run terraform apply --auto-approve, then re-run the github.com curl command. It now returns 200. This is the change process in a regulated environment: every new destination is a reviewed code change, not a manual portal edit.

Step 16: Clean Up Resources

Terminate the cluster in the Databricks UI, then destroy everything:

```
terraform destroy --auto-approve
```

This removes both resource groups and all resources in them, including the firewall. Azure deletes the managed resource group automatically.

Troubleshooting

- Cluster fails to start or times out → check that both peerings show Connected, the route's next hop matches firewall_private_ip, and the databricks-control-plane rule includes all ports. Look for Deny rows in AZFWNetworkRule.
- pip install fails → look for Deny rows in AZFWApplicationRule. The package may be downloaded from a domain your rules do not cover.
- Error about a service tag on apply → service_tag_region must have no spaces, for example WestEurope.
- Firewall fails to deploy → the subnet must be named exactly AzureFirewallSubnet and be at least /26.
- No rows in Log Analytics → wait 10 minutes and confirm the diagnostic setting diag-firewall exists on the firewall.

Outcomes

You've deployed Azure Databricks in a hub-and-spoke network with all egress controlled by Azure Firewall. You now know how to:

- Build a hub VNet with Azure Firewall and a reusable Firewall Policy.
- Allow Databricks platform traffic with service-tag network rules and package repositories with FQDN application rules.
- Connect a Databricks spoke to the hub with two-way VNet peering.
- Force all cluster egress through the firewall with a 0.0.0.0/0 UDR.
- Capture firewall decisions in Log Analytics and query them with KQL.
- Prove from a live cluster that approved destinations work and everything else is denied.
- Open a new destination through a reviewed Terraform change instead of a manual edit.

This sets the foundation for combining the hub with Private Link (front-end endpoints and DNS in the hub) and for connecting on-premises networks through a VPN or ExpressRoute gateway.